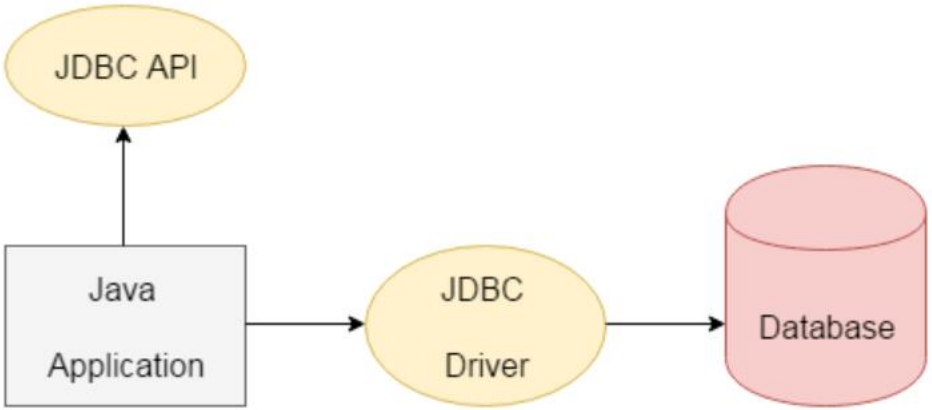
JDBC

What is JDBC

- JDBC stands for Java Database Connectivity; is a Java-based API that allows Java applications to interact with databases. It provides methods to connect to a database, execute SQL queries, and retrieve results. JDBC acts as a bridge between the Java application and the database
- It is a part of JavaSE (Java Standard Edition).
- JDBC API uses JDBC drivers to connect with the database
- There are four types of JDBC drivers: JDBC-ODBC Bridge Driver,Native Driver,Network Protocol Driver, andThin Driver



Why Should We Use JDBC

- Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

We can use JDBC API to handle database using Java program and can perform the following activities:

- Connect to the database
- Execute queries and update statements to the database
- Retrieve the result received from the database.

JDBC API

- The **java.sql** package contains classes and interfaces for JDBC API.

List of popular interfaces

- Driver interface
- Connection interface
- Statement interface
- PreparedStatement interface
- CallableStatement interface
- ResultSet interface
- ResultSetMetaData interface
- DatabaseMetaData interface
- RowSet interface

list of popular Classes

- DriverManager class
- Blob class
- Clob class
- Types class

JDBC drivers

- **JDBC** drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand. JDBC drivers are the software components which implements interfaces in JDBC APIs to enable java application to interact with the database.
- There are four types of JDBC drivers defined by Sun Microsystem that are mentioned below:

1. Type-1 driver or JDBC-ODBC bridge driver
2. Type-2 driver or Native-API driver
3. Type-3 driver or Network Protocol driver
4. Type-4 driver or Thin driver

JDBC-ODBC bridge driver – Type 1 driver

- Type-1 driver or JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. Type-1 driver is also called Universal driver because it can be used to connect to any of the databases.
- Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

JDBC-ODBC bridge driver – Type 1 driver

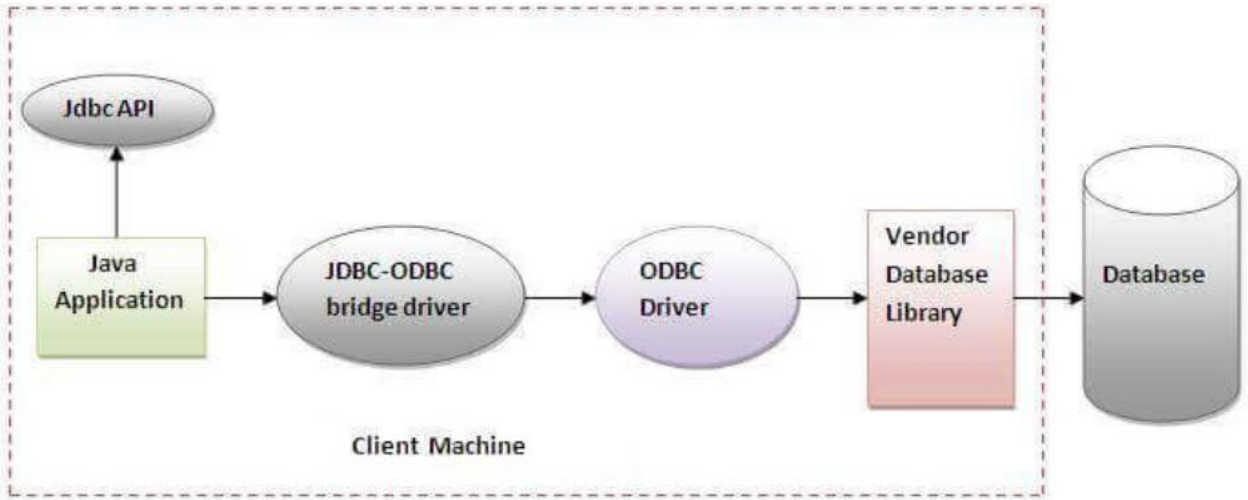


Figure- JDBC-ODBC Bridge Driver

Native-API driver

- The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

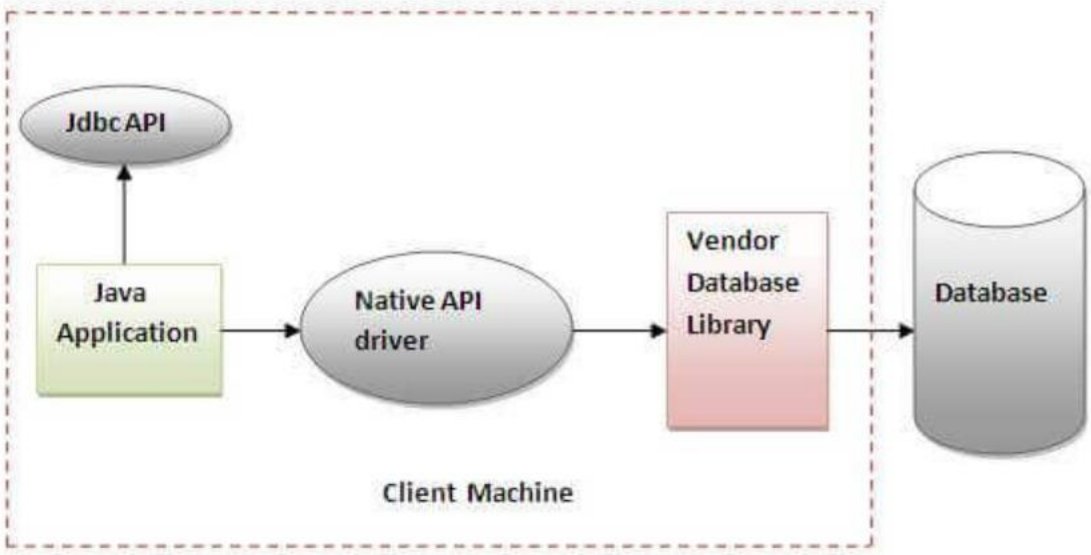
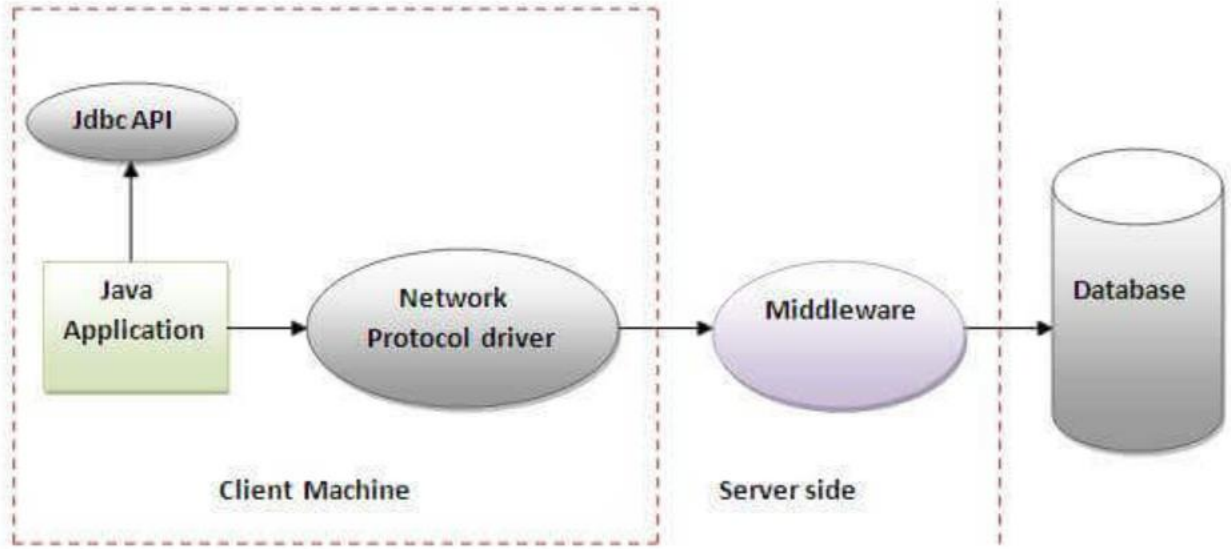


Figure- Native API Driver

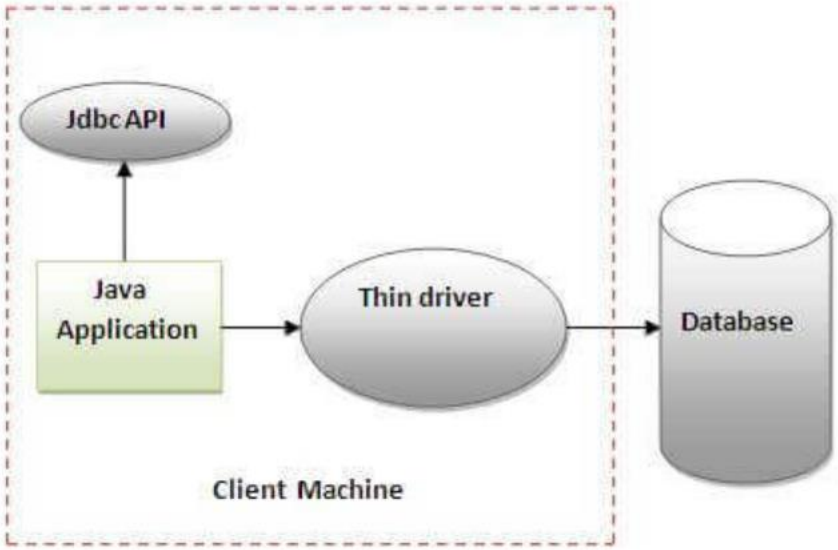
Network Protocol driver

- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.



Thin driver

- The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.



Java Database Connectivity

There are 7 steps to connect any java application with the database using JDBC. These steps are as follows:

- Import the package for JDBC API
- Load and Register the Driver class
- Create connection
- Create statement
- Execute queries
- Process the result
- Close connection

Java Database Connectivity step by step

1. Register the driver class

- The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax of forName() method

```
public static void forName(String className)throws ClassNotFoundException
```

Example to register the OracleDriver class

Here, Java program is loading oracle driver to esteblish database connection.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Java Database Connectivity step by step

- 2. Create the connection object
- The **getConnection()** method of DriverManager class is used to establish connection with the database.

Syntax of getConnection() method

```
1) public static Connection getConnection(String url)throws SQLException
2) public static Connection getConnection(String url,String name,String password)
throws SQLException
```

Example to establish connection with the Oracle database

```
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","password");
```

Java Database Connectivity step by step

- 3. Create the Statement object
- The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database..

Syntax of createStatement() method

```
public Statement createStatement()throws SQLException
```

Example to create the statement object

```
Statement stmt=con.createStatement();
```

Java Database Connectivity step by step

- 4. Execute the query
- The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of

Syntax of executeQuery() method

```
public ResultSet executeQuery(String sql)throws SQLException
```

Example to execute query

```
ResultSet rs=stmt.executeQuery("select * from emp");

while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```